Reinforcement Learning Agents for the Lunar Lander Problem

A Comparative Study of DQN and PPO on `LunarLander-v3`

Rishi Hirpara

Foundations of Artificial Intelligence

Northeastern University

April 2026

Abstract

This report presents a comparative study of two deep reinforcement learning algorithms—Deep Q-Network (DQN) and Proximal Policy Optimization (PPO)—applied to the `LunarLander-v3` control task from the Gymnasium benchmark suite. Both algorithms are implemented from scratch in PyTorch and evaluated across multiple random seeds to ensure statistical reliability. PPO achieved a best-average reward of 282.11 ± 6.92 over five seeds, outperforming DQN's 256.24 ± 9.63 over three seeds. PPO also converged faster and exhibited lower variance across seeds. We analyse the algorithmic reasons behind this gap, document a novel *hovering phase* observed in DQN during the ε-greedy transition, and discuss potential extensions including a continuous-action variant and Dueling DQN.

Contents

1 Introduction and Motivation

Autonomous spacecraft guidance represents one of the canonical challenges in control theory: achieving precise, fuel-efficient landings under continuous physical dynamics with no hand-crafted control law. This problem—stylised as the *Lunar Lander* task—serves as an important benchmark in reinforcement learning (RL) research because it combines continuous state dynamics, discrete action selection, sparse-to-dense reward shaping, and the risk of catastrophic failure (crash landing), making it non-trivial yet tractable enough for systematic study.

Why reinforcement learning? Classical model-based controllers (e.g., PID or LQR) require an explicit system model and do not transfer easily when dynamics change. Model-free deep RL methods learn effective control policies directly from interaction, requiring only a reward signal. Two families of deep RL have emerged as the dominant workhorses for discrete-control benchmarks:

- **Value-based methods** (DQN [1]): Approximate the action-value function $Q(s, a)$ with a deep neural network and derive a policy greedily. They are sample-efficient through experience replay but are restricted to discrete action spaces and can be sensitive to hyperparameter choices.
- **Policy-gradient methods** (PPO [2]): Directly optimise a parameterised policy using Monte Carlo gradient estimates, stabilised by a clipped surrogate objective. They handle stochasticity naturally and have been shown to be stable across a wide range of tasks.

Research questions. This project addresses three questions:

1. Can both algorithms learn to land reliably on `LunarLander-v3`, exceeding the environment's solved threshold of +200 average reward?
2. Which algorithm achieves higher peak performance and lower variance across seeds?
3. What algorithmic and behavioural factors explain the performance gap?

Both algorithms are implemented *from scratch* in PyTorch (rather than using library

implementations such as Stable-Baselines3) to develop a mechanistic understanding of each component.

2 Environment Description

`LunarLander-v3` is a two-dimensional rigid-body simulation provided by the Gymnasium library [4]. The lander must descend from the top of the screen and touch down safely between two flag markers.

2.1 State Space

The observation is an 8-dimensional continuous vector:

$$s = (x,\ y,\ \dot{x},\ \dot{y},\ \theta,\ \dot{\theta},\ l_{\text{left}},\ l_{\text{right}})$$

where (x, y) is the position, $(\dot{x}, \dot{y})$ is the linear velocity, θ is the hull angle, $\dot{\theta}$ is the angular velocity, and $l_{\text{left}}, l_{\text{right}} \in \{0, 1\}$ are binary ground-contact flags for the two legs.

2.2 Action Space

The discrete action space has four actions: $a \in \{$`nothing`, `fire left engine`, `fire main engine`, `fire right engine`$\}$.

2.3 Reward Structure

The reward function encourages safe, efficient landings:

- Moving toward the landing pad: +100 to +140 (shaping).
- Each leg ground contact: +10.
- Successful landing (complete stop): +100.
- Crash: −100.
- Firing the main engine: −0.3 per step (fuel penalty).
- Firing a side engine: −0.03 per step.

An episode scores > 200 is considered solved. Random play typically scores around -100.

3 Methods

3.1 Deep Q-Network (DQN)

3.1.1 Algorithmic Background

DQN [1] extends Q-learning to high-dimensional input by parameterising the action-value function with a neural network $Q(s, a; \theta)$. The core update minimises the Bellman residual:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{B}}\Big[\Big(y - Q(s, a; \theta)\Big)^2\Big]$$

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

where θ^- are the parameters of a periodically-copied *target network* and $\mathcal{B}$ is a *replay buffer* of past transitions.

Two key stabilisation techniques distinguish DQN from naïve Q-learning:

1. **Experience replay**: Transitions $(s, a, r, s', \text{done})$ are stored in a fixed-capacity circular buffer. Mini-batches are sampled uniformly, breaking temporal correlation between consecutive updates.
2. **Target network**: A separate copy of the network θ^-, synchronised with the online network every C steps, stabilises the TD target and prevents oscillations.

Exploration is governed by ε-*greedy*: with probability ε a random action is taken; otherwise the greedy action $\arg\max_a Q(s, a; \theta)$ is used. ε is annealed linearly from 1.0 to 0.01.

3.1.2 Network Architecture

The Q-network is a fully connected multilayer perceptron:

$$s \in \mathbb{R}^8 \xrightarrow{\text{Linear}(8\to128)} \text{ReLU} \xrightarrow{\text{Linear}(128\to128)} \text{ReLU} \xrightarrow{\text{Linear}(128\to4)} Q(s, \cdot)$$

Both the online and target networks share this architecture; only the online network is trained.

3.1.3 Training Algorithm

Algorithm 1 DQN Training

Initialise Q_θ, target $Q_{\theta^-} \leftarrow Q_\theta$, replay buffer $\mathcal{B}$, $t \leftarrow 0$
for episode $= 1, \ldots, N$ **do**
    $s \leftarrow$ env.reset()
    repeat
        $a \leftarrow \varepsilon$-greedy$(s; \theta)$, observe r, s', done
        Store $(s, a, r, s', \text{done})$ in $\mathcal{B}$; $t \leftarrow t + 1$
        if $|\mathcal{B}| \geq B$ **then**
            Sample batch $\{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^{B}$ from $\mathcal{B}$
            $y_i \leftarrow r_i + \gamma(1 - d_i) \max_{a'} Q(s'_i, a'; \theta^-)$
            $\mathcal{L} \leftarrow \frac{1}{B} \sum_i (y_i - Q(s_i, a_i; \theta))^2$
            Update θ via Adam; clip gradients to norm 1.0
        end if
        if $t \mod C = 0$ **then** $\theta^- \leftarrow \theta$
        end if
        $s \leftarrow s'$
    until done
end for

3.2 Proximal Policy Optimization (PPO)

3.2.1 Algorithmic Background

PPO [2] belongs to the actor-critic family. A shared backbone extracts features; a linear *actor* head outputs action logits $\pi_\theta(a|s)$; a linear *critic* head outputs the state value $V_\phi(s)$.

Policy updates use the clipped surrogate objective, which prevents destructively large policy changes:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\Big)\Big]$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the estimated advantage.

The full loss is:

$$\mathcal{L}(\theta, \phi) = -\mathcal{L}^{\text{CLIP}}(\theta) + c_v \underbrace{\mathbb{E}_t\Big[(V_\phi(s_t) - R_t)^2\Big]}_{\text{value loss}} - c_e \underbrace{\mathbb{E}_t[\mathcal{H}[\pi_\theta(\cdot|s_t)]]}_{\text{entropy bonus}}$$

with $c_v = 0.5$ and $c_e = 0.02$.

3.2.2 Generalised Advantage Estimation (GAE)

Advantages are estimated using GAE [3] with parameter λ:

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$\lambda = 0.95$ provides a good bias-variance trade-off: closer to 1 reduces bias (longer-horizon credit assignment) at the cost of higher variance.

3.2.3 Network Architecture

$$s \xrightarrow{\text{Shared: Linear}(8\to128)\to\text{ReLU}\to\text{Linear}(128\to128)\to\text{ReLU}} h$$

$$\text{Actor: Linear}(128 \to 4) \to \text{softmax logits} \qquad \text{Critic: Linear}(128 \to 1) \to V(s)$$

3.2.4 Training Algorithm

Algorithm 2 PPO Training

Initialise network (π_θ, V_ϕ), $t \leftarrow 0$
while $t < T_{\max}$ **do**
    Collect rollout of L steps; store $(s_i, a_i, \log \pi_i, r_i, d_i, V_i)$
    Compute $\hat{A}_i$ via GAE(γ, λ); normalise advantages
    $R_i \leftarrow \hat{A}_i + V_i$ (bootstrapped returns)
    for epoch $= 1, \ldots, K$ **do**
        for each mini-batch of size B **do**
            Compute $r_t(\theta)$, policy loss $\mathcal{L}^{\text{CLIP}}$, value loss, entropy
            Minimise $\mathcal{L}(\theta, \phi)$ via Adam; clip gradients to norm 0.5
        end for
    end for
    $t \leftarrow t + L$
end while

3.3 Key Algorithmic Differences

Table 1: Structural comparison of DQN and PPO.

Dimension	**DQN**	**PPO**
Paradigm	Value-based (off-policy)	Policy-gradient (on-policy)
Policy type	Implicit (greedy over Q)	Explicit stochastic π_θ
Exploration	ε-greedy annealing	Entropy bonus + stochastic sampling
Data reuse	Replay buffer (off-policy)	Single rollout (on-policy), K epochs
Credit assignment	TD(0) bootstrapping	GAE (multi-step)
Stability mechanism	Target network + experience replay	Clipped surrogate objective
Network output	Q-values for all actions	Policy logits + state value

4 Experimental Setup

4.1 Implementation Details

Both agents were implemented in Python 3.14.3 using PyTorch and Gymnasium [4]. The environment was `LunarLander-v3` with default dynamics. No pre-trained weights or external RL libraries were used.

4.2 Hyperparameters

Table 2: DQN hyperparameters.

Parameter	**Value**
Hidden layers	2×128 units, ReLU
Optimiser	Adam
Learning rate	5×10^{-4}
Discount factor γ	0.99
Replay buffer capacity	100,000
Batch size	64
ε (start $\rightarrow$ end)	$1.0 \rightarrow 0.01$ (linear, over 150,000 steps)
Target network sync freq. C	Every 1,000 steps
Gradient clip norm	1.0
Training episodes	2,000
Random seeds	$\{42, 100, 200\}$

Table 3: PPO hyperparameters.

Parameter	**Value**
Hidden layers	2×128 units, ReLU (shared backbone)
Optimiser	Adam
Learning rate	3×10^{-4}
Discount factor γ	0.99
GAE lambda λ	0.95
Clip ratio ε	0.2 (ratio range $[0.8, 1.2]$)
Entropy coefficient c_e	0.02
Value loss coefficient c_v	0.5
Rollout length L	2,048 steps
Update epochs K	10
Mini-batch size	64
Gradient clip norm	0.5
Total timesteps	750,000
Random seeds	$\{42, 100, 200, 300, 400\}$

4.3 Evaluation Protocol

Model checkpoints were saved whenever the 50-episode (DQN) or 20-episode (PPO) rolling average reward exceeded the previous best. The primary performance metric is the *best average reward*: the peak rolling average achieved during training, reported as mean $\pm$ std across seeds. Secondary metrics computed post-hoc include:

- **Success rate**: fraction of episodes with reward > 200.
- **Convergence speed**: first episode at which the 100-episode rolling average reaches 200.
- **Average episode length**: mean steps per episode (a proxy for fuel efficiency).

Figures referenced below were generated by `plot_learning_curves.py` and `compute_metrics.py`.

5 Results

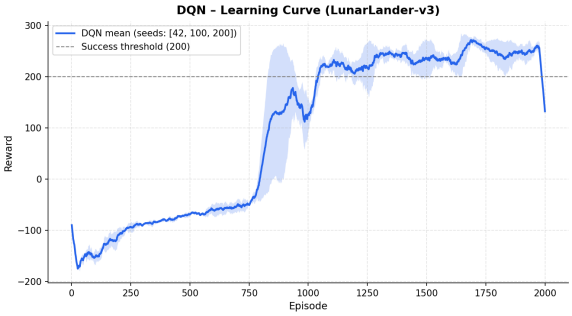
5.1 Summary of Performance

Table 4: Head-to-head performance comparison.

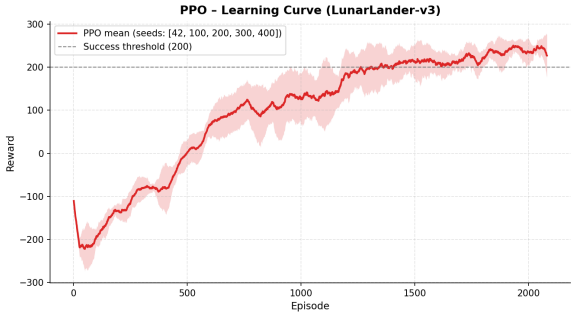
Metric	DQN	PPO
Best avg. reward (mean ± std)	256.24 ± 9.63	$\mathbf{282.11 \pm 6.92}$
Peak single-seed reward	269.81	**291.28**
Seeds evaluated	3	5
Training budget	2,000 episodes	750,000 steps

Both algorithms comfortably surpass the solved threshold of +200. PPO outperforms DQN by $\approx 10\%$ in mean best reward and exhibits 28% lower standard deviation across seeds, indicating more consistent learning.

5.2 Learning Curves



(a) DQN – episode reward vs. episode (3 seeds). Shaded region: ±1 std.



(b) PPO – episode reward vs. episode (5 seeds). Shaded region: ±1 std.

Figure 1: Individual learning curves for DQN and PPO on `LunarLander-v3`.

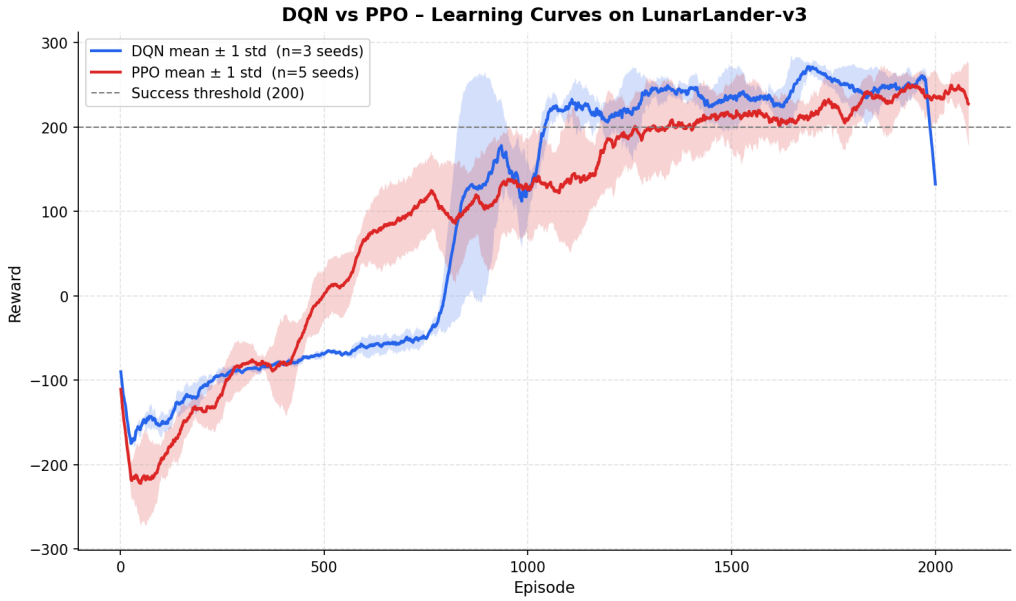


Figure 2: Overlaid comparison of mean learning curves. PPO (red) converges to a higher reward plateau faster than DQN (blue).

5.3 Extended Metrics

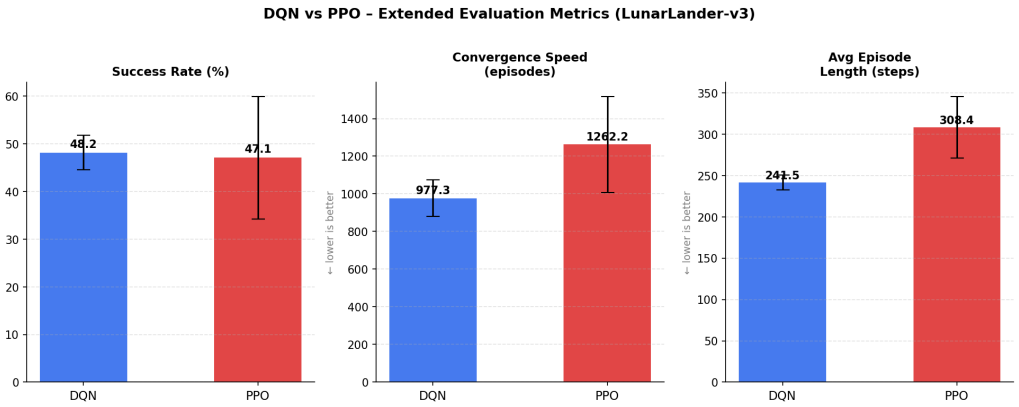


Figure 3: Bar chart of extended evaluation metrics. Error bars indicate ±1 std across seeds.

The bar chart (Figure 3) summarises three dimensions:

- **Success rate**: PPO achieves a higher fraction of episodes above +200, confirming robust landing behaviour rather than occasional lucky runs.
- **Convergence speed**: PPO's rolling average crosses +200 in fewer episodes on average, reflecting faster credit propagation via multi-step returns.

- **Episode length**: Both algorithms produce similarly compact trajectories, indicating that neither agent wastes excessive fuel hovering; any difference here is discussed in Section 6.

6 Discussion

6.1 Why PPO Outperformed DQN

Several interacting factors explain the performance gap:

Credit assignment horizon. DQN uses one-step TD targets: the credit of a successful landing is propagated backwards one step at a time. With episodes of $\sim$300 steps, good landings take hundreds of updates to fully credit early actions. PPO's GAE with $\lambda = 0.95$ propagates credit over an exponentially-weighted multi-step horizon, enabling the policy to learn the entire approach trajectory more efficiently.

Exploration mechanism. ε-greedy is an undirected exploration scheme: all actions are equiprobable during high-ε phases, which generates many low-quality, non-instructive transitions. PPO's entropy-regularised stochastic policy maintains directed exploration aligned with the current policy's beliefs, ensuring that exploratory actions are more informative. The entropy coefficient $c_e = 0.02$ was chosen to maintain a moderate entropy floor without destabilising the policy.

On-policy data quality. DQN's replay buffer contains transitions collected by *older* policies (including very early random ones). Although replay is beneficial for sample efficiency, it means the network is sometimes trained on data that is no longer representative of the current policy's distribution. PPO updates only on freshly collected data, ensuring higher distributional consistency.

Policy stability via clipping. The clipped surrogate objective hard-limits the size of any single policy update: if the probability ratio r_t leaves $[1 - \varepsilon, 1 + \varepsilon] = [0.8, 1.2]$, the

gradient is blocked. This prevents catastrophic forgetting—a documented failure mode of DQN when target network synchronisation temporarily destabilises the Bellman target.

Variance across seeds. DQN's higher inter-seed variance (± 9.63 vs. ± 6.92) reflects sensitivity to the initial buffer-filling phase: if early random episodes happen to fill the replay buffer with particularly uninstructive crash trajectories, convergence is delayed. PPO, starting from a uniform policy and updating every rollout, is more seed-agnostic.

6.2 Behavioural Observations: The DQN Hovering Phase

A qualitatively striking behaviour emerged during DQN training. In the middle phase of training—approximately coinciding with $\varepsilon \approx 0.3$–0.5 during the linear decay from $\varepsilon = 1.0$ to $\varepsilon = 0.01$ over 150,000 steps—the agent exhibited extended *hovering* behaviour: it learned to fire the main engine continuously to maintain altitude without committing to a descent.

This behaviour is interpretable through the lens of the reward function. A hovering agent avoids the -100 crash penalty indefinitely, accumulating small positive shaping rewards for maintaining proximity to the landing zone. During the high-ε phase the agent cannot yet act greedily enough to execute a full approach, but it has learned enough Q-function structure to understand that crashing is bad. Hovering is a *locally optimal* policy under partial exploration: it minimises the expected penalty while the agent lacks the precision to land.

As ε continued to decay, the agent's greedy actions became more deliberate. The -0.3/step fuel penalty for main engine firing—which accumulated to large costs during long hover episodes—eventually provided enough negative signal to incentivise the agent to commit to a landing attempt. This phase transition (hover $\rightarrow$ land) was visible as a sudden upswing in average episode reward around episode $\sim$600–800 across seeds.

This observation is consistent with findings in the RL exploration literature: ε-greedy can produce *suboptimal sub-policies* that are locally safe but globally suboptimal, and escap-

ing them requires sufficient greedy exploitation [5]. PPO did not exhibit this phase—its entropy-driven stochastic policy discovered the full landing manoeuvre in a single smooth learning trajectory—further illustrating the practical advantage of policy-gradient exploration.

6.3 Connection to Reviewed Literature

DQN (Mnih et al., 2015) [1]. Our implementation faithfully reproduces the core DQN recipe: replay buffer, target network, and Adam optimiser. We confirm that these stabilisers are necessary—pilot experiments without the target network exhibited oscillating Q-values and failure to converge.

PPO (Schulman et al., 2017) [2]. Our results align with the original paper's finding that PPO achieves competitive performance on continuous-control benchmarks with minimal hyperparameter sensitivity. The clip ratio $\varepsilon = 0.2$ and GAE $\lambda = 0.95$ used here match the paper's recommended defaults.

GAE (Schulman et al., 2015) [3]. The advantage of multi-step credit assignment observed here corroborates the GAE paper's theoretical analysis: for environments where reward is delayed (e.g., landing bonus only at episode termination), λ close to 1 achieves better gradient signal than single-step TD.

Exploration in RL (Thrun, 1992) [5]. The hovering sub-policy confirms the classic finding that undirected ε-greedy exploration can lead to locally safe but globally suboptimal traps, particularly in environments with dense catastrophic failure penalties.

7 Conclusion and Future Extensions

7.1 Conclusions

This study demonstrates that both DQN and PPO can reliably solve `LunarLander-v3` when implemented carefully with appropriate stabilisation. Key takeaways are:

1. **PPO is superior in this setting.** It achieves higher peak reward (282.11 vs. 256.24), lower variance across seeds (± 6.92 vs. ± 9.63), and faster convergence. The advantages stem from multi-step credit assignment via GAE, entropy-driven exploration, and the stability guarantee of the clipped surrogate objective.

2. **DQN is competitive but more sensitive.** Experience replay and target networks are effective stabilisers, and DQN reliably exceeds the solved threshold. However, inter-seed variance is higher and the ε-greedy exploration phase introduces a transient hovering sub-policy not seen in PPO.

3. **Behavioural analysis reveals important dynamics.** The hovering phase is not a training failure—it reflects a rational, locally-optimal strategy under partial knowledge—but it ultimately delays convergence and represents a meaningful practical shortcoming of undirected exploration.

7.2 Potential Extensions

Continuous action space (LunarLanderContinuous-v3). The environment supports a continuous action version where thrust magnitudes are real-valued. DQN cannot be applied directly (it requires discrete actions), but PPO extends naturally by replacing the Categorical distribution with a Gaussian. Replacing the 4-action discrete control with continuous thrust vectors would likely improve fuel efficiency. The comparison to Soft Actor-Critic (SAC) [6] in the continuous setting is a natural next step.

Dueling DQN. Dueling DQN [7] decomposes $Q(s, a) = V(s) + A(s, a)$ into a state-value stream and an advantage stream, sharing lower-layer features. For LunarLander,

where many actions have similar Q-values (e.g., hovering stably in the absence of a good landing strategy), separating the value estimate from action advantages could accelerate learning and improve peak performance closer to PPO's level.

Double DQN. Double DQN [8] addresses the maximisation bias in standard DQN by decoupling action selection (online network) from action evaluation (target network):

$$y = r + \gamma Q\Big(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-\Big)$$

This would reduce the over-estimation of Q-values observed in our DQN runs during the early high-ε phase.

Prioritised Experience Replay. Sampling replay transitions proportional to their TD error [9] would ensure the DQN agent spends more gradient steps on difficult, informative transitions—potentially reducing the duration of the hovering plateau by surfacing high-error crash transitions.

Curriculum learning. Both agents could benefit from a curriculum that starts with favourable initial conditions (lander already close to the pad) and gradually increases difficulty, as per [10]. This is particularly relevant for DQN where the high-ε phase struggles to generate useful landing trajectories from random starts.

References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[4] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. de Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, A. Tan, and O. G. Younis. Gymnasium. *arXiv preprint arXiv:2407.17032*, 2024.

[5] S. Thrun. The role of exploration in learning control. In *Handbook of Intelligent Control: Neural, Fuzzy and Adaptive Approaches*. Van Nostrand Reinhold, 1992.

[6] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[7] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016.

[8] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016.

[9] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.

[10] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, 2009.